

FH JOANNEUM - University of Applied Sciences

Dynamic Resource Allocation and Scaling of Stateful Workloads in Kubernetes

Bachelorarbeit

zur Erlangung des akademischen Grades

„Bachelor of Science in Engineering“

eingereicht am Studiengang Wirtschaftsinformatik

Studienrichtung: Wirtschaftsinformatik

Verfasst von: Johann Krischan Hipolito

Betreuung: Michael Nestler

Graz, 2026

Table of Contents

Table of Contents	i
List of Figures	iii
List of Tables	iv
List of Abbreviations	v
List of Symbols	vi
List of Formulas	vii
Kurzfassung	viii
Abstract	viii
1 Introduction	1
1.1 Cloud Computing and the Rise of Container Orchestration	1
1.2 Kubernetes as the Standard for Container Orchestration	1
1.3 The Challenge of Scaling Stateful Applications	2
1.4 Autoscaling Strategies: From Reactive to Predictive	2
1.5 Research Objective and Thesis Structure	3
2 Scaling Stateful and Stateless Applications in Kubernetes	4
2.1 Stateless Applications and Horizontal Scalability	4
2.2 The Challenges of Scaling Stateful Applications	4
3 Hypothesis: The Operator Pattern with KEDA as a Superior Autoscaling Strategy for Stateful Workloads	7
3.1 The Operator Pattern as a Foundation for Stateful Automation.	7
3.2 KEDA's Event-Driven Model as a Proactive Trigger.	7
3.3 Supporting Evidence from the Literature.	8
3.4 Scope and Expected Outcome.	8
4 Technologies	9
4.1 K3s	9
4.2 FastAPI	9
4.3 Redis Cluster	9
4.4 Horizontal Pod Autoscaler (HPA)	10
4.5 KEDA	10
4.6 Prometheus and Grafana	10
4.7 Helm	11
4.8 Grafana k6	11
5 Experiment Architecture	12
5.1 Components Outside the Cluster	12
5.2 Components Inside the Cluster	15
5.2.1 The default Namespace: FastAPI Entrypoint	15
5.2.2 The redis Namespace: Stateful Data Tier and Scaling Configurations	16
5.2.3 The ot-operators and keda Namespaces: Scaling Controllers	20
5.2.4 The monitoring Namespace: Observability Stack	21
5.3 End-to-End Data Flow	21
6 Experimental Test Flow	23
6.1 Scenario A: Native HPA on a Self-Managed Redis Cluster	23
6.2 Scenario B: Operator and KEDA on an Operator-Managed Redis Cluster	24
6.3 The Necessity of Quiescing the Load Generator Before Scale-In	25

7	Results and Analysis	26
8	Conclusion	27
	Anhang	28
	Glossar	29
	Literaturverzeichnis	30
	Stichwortverzeichnis	33
	Anlagen	34

List of Figures

1	High-level architecture of the testbed.	12
---	---	----

List of Tables

List of Abbreviations

API	Application Programming Interface
YAML	YAML Ain't Markup Language
KEDA	Kubernetes Event-Driven Autoscaling
CRD	Custom Resource Definition
HTTP	Hypertext Transfer Protocol
HPA	Horizontal Pod Autoscaler
JS	JavaScript

List of Symbols

Hier steht das Symbolverzeichnis.

List of Formulas

Hier steht das Formelverzeichnis.

Kurzfassung

Hier steht die Kurzfassung auf Deutsch.

Abstract

Here is the abstract in English.

1 Introduction

1.1 Cloud Computing and the Rise of Container Orchestration

The rapid evolution of modern software systems has fundamentally transformed the way infrastructure is designed, deployed, and operated. At the heart of this transformation lies *cloud computing* — a paradigm that enables on-demand access to a shared pool of configurable computing resources, including networks, servers, storage, applications, and services, that can be rapidly provisioned and released with minimal management effort (Mell and Grance, 2011). This model has shifted the industry away from static, on-premises data centres toward elastic, scalable environments that can react dynamically to workload demands.

As organisations increasingly adopt cloud-native architectures, the notion of designing systems that are resilient, scalable, and operationally excellent has become central to engineering practice. The AWS Well-Architected Framework, for instance, codifies these concerns into a set of established design principles for container-based workloads, emphasising repeatability, automation, and observability as first-class concerns in modern infrastructure (Amazon Web Services, 2022).

A critical enabler of cloud-native architectures is the *container* — a lightweight, self-contained unit of software that packages application code together with its dependencies, configuration, and runtime environment. Unlike traditional virtual machines, containers share the host operating system kernel and therefore incur significantly less overhead while still providing process-level isolation (Docker Inc., 2016). This characteristic makes containers especially well suited to microservice architectures, where large monolithic applications are decomposed into smaller, independently deployable services that can be scaled individually based on demand.

1.2 Kubernetes as the Standard for Container Orchestration

Managing containerised workloads at scale introduces significant operational complexity. Scheduling containers across a fleet of machines, maintaining desired application state, performing rolling updates, and recovering from failures all require robust automation. It is in response to exactly these challenges that *Kubernetes* — also referred to as K8s — emerged as the dominant container orchestration platform.

Originally designed and used internally at Google as the *Borg* system, Kubernetes was open-sourced in 2014 and donated to the Cloud Native Computing Foundation (CNCF), where it has since grown into the second-largest open-source project in the world (IBM, 2024). Kubernetes provides a declarative Application Programming Interface (API) for defining the desired state of an application cluster; its control plane continuously reconciles the observed state of the system against this desired state, automatically scheduling pods, managing service discovery, and orchestrating storage (Kubernetes Documentation, 2024d).

To reduce operational complexity further, lightweight Kubernetes distributions such

as K3s — developed by Rancher and certified by the CNCF — have gained widespread adoption. K3s packages the full Kubernetes feature set into a single binary with a significantly reduced memory footprint, making it suitable for edge computing, IoT environments, and resource-constrained infrastructure (K3s Project, 2024). In the context of this thesis, K3s serves as the underlying cluster management layer for the experimental infrastructure under investigation.

1.3 The Challenge of Scaling Stateful Applications

While Kubernetes excels at managing stateless workloads — where any replica of a service can handle any request independently — scaling *stateful applications* presents a distinct set of challenges. Stateful applications, such as in-memory databases and message queues, maintain persistent data or internal state that must be carefully managed across the lifecycle of individual pods. Kubernetes provides the `StatefulSet` primitive to address these requirements, offering stable network identities, ordered deployment and scaling, and persistent volume binding (Kubernetes Documentation, 2024f).

Redis is a canonical example of such a stateful application. It is a high-performance, in-memory data store widely used for caching, session management, and real-time analytics (Redis Ltd., 2024a). When deployed as a Redis Cluster across multiple nodes, it uses sharding to distribute data and replication to ensure fault tolerance. Scaling a Redis Cluster in a Kubernetes environment is non-trivial: adding or removing nodes requires coordinated resharding of data slots, careful management of cluster topology, and maintenance of quorum — operations that introduce significant risk if performed reactively under resource pressure.

Recent work from the CNCF community has highlighted these concerns, noting that while Kubernetes operators can partially automate lifecycle management for Redis, the path toward robust, cloud-native stateful services remains an open challenge (Cloud Native Computing Foundation, 2024).

1.4 Autoscaling Strategies: From Reactive to Predictive

Kubernetes provides native support for horizontal scaling through the *Horizontal Pod Autoscaler* (HPA), which monitors CPU and memory utilisation metrics and adjusts the number of pod replicas accordingly (Kubernetes Documentation, 2024a). While the HPA is effective for stateless services under gradually increasing load, its reactive nature introduces an inherent latency: scaling decisions are made only *after* resource thresholds have already been breached. For latency-sensitive or stateful workloads, this lag can result in degraded performance, pod restarts, or cascading failures during traffic spikes (Thoras, 2024).

An increasingly popular alternative is *Kubernetes Event-Driven Autoscaling* (Kubernetes Event-Driven Autoscaling (KEDA)), a CNCF-graduated project that extends the standard HPA to support event- and metric-driven scaling from a rich ecosystem of external sources, including message queues, databases, and custom Prometheus metrics (KEDA

Project, 2024a; Cloud Native Computing Foundation, 2023). By decoupling scaling decisions from simple resource utilisation and enabling them to respond to application-level events, KEDA enables a more proactive and context-aware approach to scaling. This is particularly relevant for stateful workloads where pre-emptive resource provisioning may prevent the cluster-level disruptions that reactive scaling cannot avoid.

1.5 Research Objective and Thesis Structure

This thesis investigates the performance differences between two autoscaling strategies for stateful applications deployed in a Kubernetes cluster managed by K3s. The first strategy employs the native Horizontal Pod Autoscaler in a *reactive* configuration, scaling a Redis Cluster in response to CPU and memory resource expenditure. The second strategy employs a *custom KEDA-based operator* in a predictive, event-driven configuration, triggering scaling decisions based on application-level metrics sourced from a Prometheus observability stack.

The experimental infrastructure consists of a FastAPI-based endpoint service (FastAPI, 2024b), a Redis Cluster with a minimum of three nodes, an observability stack comprising Prometheus and Grafana deployed via Helm, and two scaling configuration manifests representing the naive HPA and KEDA-driven approaches respectively. All components are deployed on a K3s cluster, providing a reproducible and resource-constrained environment representative of real-world edge and on-premises deployments.

The remainder of this thesis is structured as follows. Chapter 2 details the differences between the automatic provisioning of stateless and stateful applications, as well as the issues that arise when scaling stateful workloads. Chapter 4 provides the theoretical background on Kubernetes architecture, autoscaling mechanisms, stateful workloads, and the Redis Cluster model. Chapter 5 describes the system design and experimental setup. Chapter 7 presents the experimental results and performance analysis. Chapter 8 summarises the findings and discusses directions for future work.

2 Scaling Stateful and Stateless Applications in Kubernetes

A fundamental distinction in distributed systems design is that between *stateless* and *stateful* applications, and this distinction has direct consequences for how workloads are deployed, managed, and scaled inside a Kubernetes cluster. Understanding this divide is essential for motivating the core research question of this thesis.

2.1 Stateless Applications and Horizontal Scalability

A stateless application does not retain any client-specific data between requests. Each request is self-contained and can be served by any replica of the application without prior knowledge of previous interactions (Red Hat, 2025). This architectural property makes stateless services trivially scalable: because every replica is functionally identical, the Kubernetes scheduler can start or terminate any number of them at any time without coordination. Kubernetes `Deployments` and `ReplicaSets` are the canonical workload primitives for stateless applications, providing identical, interchangeable pod replicas with no notion of stable identity or ordered lifecycle (Kubernetes Documentation, 2024f).

Horizontal scaling of a stateless service via the Horizontal Pod Autoscaler (HPA) is therefore straightforward: when aggregate CPU or memory utilisation exceeds a configured threshold, the HPA controller issues a scale-out instruction to the relevant `Deployment`, and the new replicas are scheduled and become available without any data migration, topology reconfiguration, or disruption to existing replicas (Kubernetes Documentation, 2024a). Conversely, when load decreases, replicas can be terminated immediately since no in-replica state is lost. This symmetry between scale-out and scale-in is what makes the reactive HPA model well-suited to stateless workloads.

2.2 The Challenges of Scaling Stateful Applications

Stateful applications, by contrast, maintain persistent data or internal state that is specific to individual instances and must survive pod restarts, rescheduling events, and scaling operations. This category includes relational databases, key-value stores, message queues, and consensus systems. Kubernetes provides the `StatefulSet` workload primitive to accommodate these requirements. Unlike a `Deployment`, a `StatefulSet` guarantees that each pod receives a stable, unique network identity (e.g., `redis-0`, `redis-1`), a dedicated `PersistentVolumeClaim` that is bound across restarts, and an ordered, sequential lifecycle for pod creation and termination (Kubernetes Documentation, 2024f, Portworx, 2024).

These guarantees, while necessary for correctness, fundamentally complicate autoscaling. The following challenges are particularly relevant in the context of this thesis:

Ordered deployment and termination. `StatefulSet` pods are created and deleted in strict ordinal order. When scaling out from n to $n + 1$ replicas, Kubernetes will not proceed until the n -th pod is fully *Ready*. This serialisation ensures application-level invariants (such as cluster quorum) are maintained but introduces latency that can be problematic under sudden load spikes (Kubernetes Documentation, 2024f). A reactive autoscaler such as the HPA, which fires only after a resource threshold has already been breached, is therefore poorly positioned to absorb burst traffic for stateful workloads: by the time new replicas are ordered, scheduled, initialised, and joined to the cluster, the performance degradation may already be severe.

Persistent volume provisioning. Each new pod added to a `StatefulSet` requires a new `PersistentVolumeClaim` to be dynamically provisioned by the storage backend. Depending on the underlying storage class and infrastructure, this provisioning step can itself introduce non-trivial latency, further extending the time-to-ready for new stateful replicas compared to stateless counterparts (Portworx, 2024).

Cluster topology and data rebalancing. For distributed databases and in-memory stores that shard data across their nodes — such as Redis Cluster — adding or removing a node is not merely a matter of starting or stopping a process. Redis Cluster partitions its keyspace across exactly 16 384 hash slots, each of which is assigned to a primary node (Redis Ltd., 2024b). When a new node is added, the cluster operator must *reshard* a subset of hash slots from existing nodes to the new one; when a node is removed, its slots must first be migrated away. This resharding process involves live data movement between nodes and, if performed reactively under resource pressure, carries a non-trivial risk of increased latency, partial unavailability, or data inconsistency (Google Cloud, 2023).

Quorum and availability constraints. Many stateful distributed systems depend on a quorum of nodes being available to elect a leader, acknowledge writes, or service reads. Redis Cluster requires at least one primary per shard to be available, and Kubernetes `PodDisruptionBudget` (PDB) resources are the mechanism used to express minimum availability constraints during voluntary disruptions such as autoscaling events (Kubernetes Documentation, 2024b). Aggressive scale-in triggered by a reactive autoscaler can violate quorum constraints if PDBs are not carefully configured, risking full cluster unavailability.

The HPA and stateful workloads. While the HPA technically supports `StatefulSets` as scaling targets, its reactive, threshold-based model is widely considered inappropriate for database and cache workloads (Vazquez, 2026). The HPA operates on *lagging indicators*: it reacts to resource pressure that has already materialised. For a stateful application such as Redis, where high memory pressure can degrade cache hit rates and high CPU pressure can stall replication, by the time the HPA observes a breach and a new node has completed resharding, a significant window of degraded performance will have already elapsed. Furthermore, the HPA's default synchronisation period of

15 seconds (KEDA Project, 2024d) means that multiple polling cycles may pass before a scaling decision is acted upon, compounding the lag.

These challenges collectively motivate the investigation of a more *proactive*, event-driven autoscaling strategy for stateful workloads, as explored in this thesis through the use of KEDA.

3 Hypothesis: The Operator Pattern with KEDA as a Superior Autoscaling Strategy for Stateful Workloads

The limitations of the reactive HPA model described in Section 2.2 motivate the central hypothesis of this thesis: that a *proactive, event-driven autoscaling strategy*, implemented through a custom Kubernetes operator in combination with KEDA, is a superior method of scaling stateful workloads compared to the native HPA baseline.

3.1 The Operator Pattern as a Foundation for Stateful Automation.

The Kubernetes *operator pattern* was conceived precisely to address the gap between the generic automation that Kubernetes provides out of the box and the deep, application-specific knowledge required to manage complex stateful systems reliably. According to the official Kubernetes documentation, the operator pattern “aims to capture the key aim of a human operator who is managing a service or set of services”, encoding that operational expertise into a software controller that participates in the standard Kubernetes reconciliation loop (Kubernetes Documentation, 2024e). Operators extend the Kubernetes API through *Custom Resource Definitions* (CRDs) and an accompanying controller that continuously reconciles the observed state of the custom resource against its declared desired state.

Critically, the CNCF Operator White Paper explicitly acknowledges that “Kubernetes primitives were not built to manage state by default” and that “relying on Kubernetes primitives alone brings difficulty managing stateful application” lifecycle (CNCF TAG App Delivery, 2023). The operator pattern is therefore not merely a convenience — it is the architecturally recommended approach for encoding domain-specific concerns such as data resharding, replication topology maintenance, and quorum-safe scaling into a Kubernetes-native automation layer. For a Redis Cluster deployment, a custom operator is capable of expressing and enforcing exactly the application-level invariants that a generic HPA controller cannot: for example, ensuring that a scale-out event is only issued once the cluster is fully healthy, and that the requisite resharding of hash slots is coordinated atomically with the scheduling of new `StatefulSet` pods.

3.2 KEDA’s Event-Driven Model as a Proactive Trigger.

Complementing the operator pattern, KEDA introduces an event-driven trigger layer that decouples scaling decisions from raw infrastructure metrics. By consuming application-level signals exposed via Prometheus — such as request rate, cache hit ratio, or queue depth — KEDA’s `ScaledObject` can issue a scaling recommendation *before* CPU or memory thresholds are breached, effectively shifting the scaling decision upstream in the load curve (KEDA Project, 2024d). This is in direct contrast to the HPA model, in which scaling decisions are always downstream of resource exhaustion. The additional configurability of KEDA’s `ScaledObject` — including fine-grained control over polling intervals, cooldown periods, and minimum replica counts (Medium /

3. Hypothesis: The Operator Pattern with KEDA as a Superior Autoscaling Strategy for Stateful Workloads

emreblblvv, 2024) — further allows the autoscaling behaviour to be tuned to the specific performance characteristics of a Redis Cluster, where aggressive scale-in during low-traffic periods is inadvisable due to the cost of subsequent resharding.

3.3 Supporting Evidence from the Literature.

The hypothesis is supported by a growing body of academic and industry research. Mondal et al. identified that the HPA's reliance on CPU and memory as lagging indicators makes it "incapable of foreseeing upcoming workload spikes", leading to Quality of Service violations, long-tail latency, and wasted resources (Wanigasooriya and Ekanayake, 2026). Wanigasooriya and Ekanayake similarly demonstrated in the NimbusGuard study that proactive autoscaling frameworks "translate into superior performance and cost efficiency compared to existing reactive methods" when evaluated alongside standard HPA and KEDA baselines under identical, phased load patterns (Wanigasooriya and Ekanayake, 2026). Although that study positions KEDA itself as part of the reactive baseline relative to a deep-learning agent, it nevertheless confirms that event-driven, application-aware autoscaling consistently outperforms pure resource-metric scaling — a finding that directly supports the use of KEDA-sourced Prometheus metrics as the trigger mechanism in the present thesis.

Furthermore, Toka et al. identified three structural drawbacks in the current Kubernetes reactive scaling process: its purely observation-based nature, its inability to adapt to demand variability, and the burden of manually tuning numerous threshold parameters (Wanigasooriya and Ekanayake, 2026). A KEDA-based implementation driven by application-level Prometheus metrics partially addresses all three concerns: it provides an application-aware trigger rather than an infrastructure-level observation, it can be parameterised against workload-specific signals that vary with the demand pattern of the Redis Cluster, and it reduces manual threshold tuning by anchoring scaling decisions to semantically meaningful metrics.

3.4 Scope and Expected Outcome.

Given this theoretical and empirical foundation, this thesis hypothesises that the KEDA-driven operator approach will exhibit measurably lower response latency under load spikes, fewer instances of cluster-level resource saturation, and a more stable Redis Cluster topology compared to the naive HPA baseline. The experimental evaluation presented in Chapter 7 is designed to test this hypothesis directly, using Grafana k6 to generate controlled, reproducible traffic patterns against both configurations and Prometheus-backed Grafana dashboards to measure the resulting autoscaling behaviour and service performance.

4 Technologies

This section provides a brief overview of each technology that constitutes the experimental infrastructure used in this thesis. Together these components form a reproducible, cloud-native testbed for evaluating the two autoscaling strategies under investigation.

4.1 K3s

K3s is a CNCF-certified, lightweight Kubernetes distribution developed by Rancher. It packages the full Kubernetes control plane into a single binary of less than 70 MB and eliminates several heavyweight dependencies of upstream Kubernetes, making it well-suited for resource-constrained environments such as edge devices, IoT deployments, and developer workstations (K3s Project, 2024). Despite its reduced footprint, K3s is fully conformant with the Kubernetes API and supports the complete set of Kubernetes primitives required by this thesis, including *StatefulSets*, *CustomResourceDefinitions*, and Helm chart deployments. K3s serves as the cluster management layer for all infrastructure described in this thesis.

4.2 FastAPI

FastAPI is a modern, high-performance Python web framework designed for building RESTful APIs. It is built on top of *Starlette* for its web-server layer and *Pydantic* for data validation, and it uses Python type hints to enable automatic request validation and interactive OpenAPI documentation generation (FastAPI, 2024b). FastAPI is built around the Asynchronous Server Gateway Interface (ASGI), enabling non-blocking I/O operations that are especially advantageous in workloads involving concurrent database or cache interactions (FastAPI, 2024a). In the context of this thesis, a FastAPI application serves as the Hypertext Transfer Protocol (HTTP) endpoint to the cluster, receiving incoming requests from the external load generator and routing read and write operations to the Redis Cluster backend.

4.3 Redis Cluster

Redis is an open-source, in-memory data structure store widely used as a cache, session store, and real-time data platform (Redis Ltd., 2024a). The Redis Cluster topology used in this thesis distributes data across multiple primary nodes using a hash-slot-based sharding mechanism. The full keyspace is divided into 16 384 hash slots, each assigned to a primary node, and each primary is optionally backed by one or more replica nodes for fault tolerance (Redis Ltd., 2024b). Redis Cluster provides high availability through automatic failover: if a primary becomes unreachable, one of its replicas is promoted. In this thesis, the Redis Cluster is deployed as a Kubernetes *StatefulSet* with a minimum of three primary nodes and forms the core stateful workload under performance evaluation.

4.4 Horizontal Pod Autoscaler (HPA)

The Horizontal Pod Autoscaler is a built-in Kubernetes controller that automatically adjusts the number of pod replicas in a workload based on observed resource metrics, most commonly CPU and memory utilisation (Kubernetes Documentation, 2024a). The HPA periodically queries the Kubernetes Metrics Server (or a custom metrics adapter) and computes a desired replica count proportional to the ratio of current to target utilisation. Its synchronisation loop runs every 15 seconds by default. While effective for many stateless use cases, the HPA is a fundamentally *reactive* mechanism: scaling decisions are always downstream of observed resource conditions, making it vulnerable to the lag issues described in Section 2.2. In this thesis, the HPA is deployed as the *naïve* baseline scaling strategy, configured to scale the Redis *StatefulSet* in response to CPU and memory utilisation thresholds.

4.5 KEDA

Kubernetes Event-Driven Autoscaling (KEDA) is a CNCF-graduated open-source project that extends the Kubernetes HPA with the ability to scale workloads based on event sources and external metrics beyond simple CPU and memory (KEDA Project, 2024a). KEDA introduces the *ScaledObject* custom resource, which declares a scaling target (such as a *Deployment* or *StatefulSet*), a polling interval, cooldown periods, and one or more *scalers* that define the metric source (KEDA Project, 2024c). When the configured metric from the scaler exceeds a threshold, KEDA adjusts the replica count of the target workload by updating the underlying HPA on the user’s behalf. In this thesis, KEDA is configured with a Prometheus scaler that reads application-level metrics — such as request queue depth and cache hit ratio — directly from the Prometheus monitoring stack, enabling proactive, context-aware scaling decisions before resource exhaustion occurs (KEDA Project, 2024b).

4.6 Prometheus and Grafana

The observability stack in this thesis is deployed using the *kube-prometheus-stack* Helm chart, which bundles Prometheus, Grafana, and Alertmanager into a single, opinionated release (Prometheus Community, 2024a). *Prometheus* is an open-source monitoring system and time-series database that collects metrics by periodically scraping HTTP endpoints exposed by instrumented targets (Prometheus Authors, 2024). In the Kubernetes context, it discovers scrape targets automatically via Kubernetes service discovery, collecting cluster-level metrics from *kube-state-metrics* and *node-exporter*, as well as application-level metrics from the FastAPI and Redis exporters. *Grafana* is an open-source analytics and visualisation platform that queries Prometheus via its native data source integration and renders the collected time-series data as configurable dashboards (Grafana Labs, 2024b). Grafana dashboards are used throughout the experimental evaluation in this thesis to visualise autoscaling behaviour, latency distributions, and resource utilisation over time. Crucially, the Prometheus instance also acts as the external metric backend for the KEDA Prometheus scaler, creating a

unified data path from application telemetry to autoscaling decisions.

4.7 Helm

Helm is the de-facto package manager for Kubernetes. It introduces the concept of a *chart* — a collection of templated Kubernetes manifests that describe a deployable application — and a *release*, which is a running instance of a chart in a cluster (Helm Project, 2024). Helm charts enable repeatable, version-controlled deployments with configurable values overrides and built-in rollback support, significantly reducing the operational complexity of deploying multi-resource applications such as the *kube-prometheus-stack*. In this thesis, Helm is used to deploy the observability stack, and the deployment configurations are parameterised through values files that are committed alongside the thesis infrastructure code.

4.8 Grafana k6

Grafana k6 is an open-source, developer-focused performance and load testing tool (Grafana Labs, 2024c). Tests in k6 are written as JavaScript modules, and k6 executes them using a Go runtime which maps each *virtual user* (VU) to a lightweight goroutine, enabling high concurrency with low resource overhead (Grafana Labs, 2024d). k6 supports configurable test scenarios including ramp-up and ramp-down of virtual users over time, making it possible to simulate realistic traffic patterns such as gradual load increases, sustained peaks, and sudden spikes (Grafana Labs, 2024a). In the context of this thesis, k6 runs *outside* the Kubernetes cluster as an independent load generator. It sends configurable volumes of HTTP traffic through the FastAPI endpoint and into the Redis Cluster, producing the synthetic workload against which both autoscaling strategies — HPA and KEDA — are evaluated. The ability to precisely control the number of virtual users and the ramp profile makes k6 well-suited for generating reproducible, parameterised load scenarios that can isolate the performance characteristics of each scaling approach.

5 Experiment Architecture

This section describes the architecture of the experimental testbed: the physical and logical placement of every component, how they are wired together, and the path a synthetic request and its derived metrics take through the system. The two scaling strategies under investigation—the native Horizontal Pod Autoscaler (HPA) acting on a self-managed cluster, and the Opstree Redis Operator driven by Kubernetes Event-Driven Autoscaling (KEDA)—are deployed as mutually exclusive, interchangeable modules in an otherwise identical pipeline, so that any measured difference can be attributed solely to the scaling mechanism. The procedure that exercises this architecture, together with the associated hypotheses, is presented separately in Section 6; the conceptual description of each individual technology is given in Section 4.

Figure 1 gives a high-level overview. A clear boundary separates the single component that lives *outside* the Kubernetes cluster—the load generator on an intermediary virtual machine—from everything deployed *inside* the six-node cluster. The remainder of this section follows that division: Section 5.1 covers the external load generator, and Section 5.2 covers the in-cluster components, organised by the Kubernetes namespace in which they are deployed.

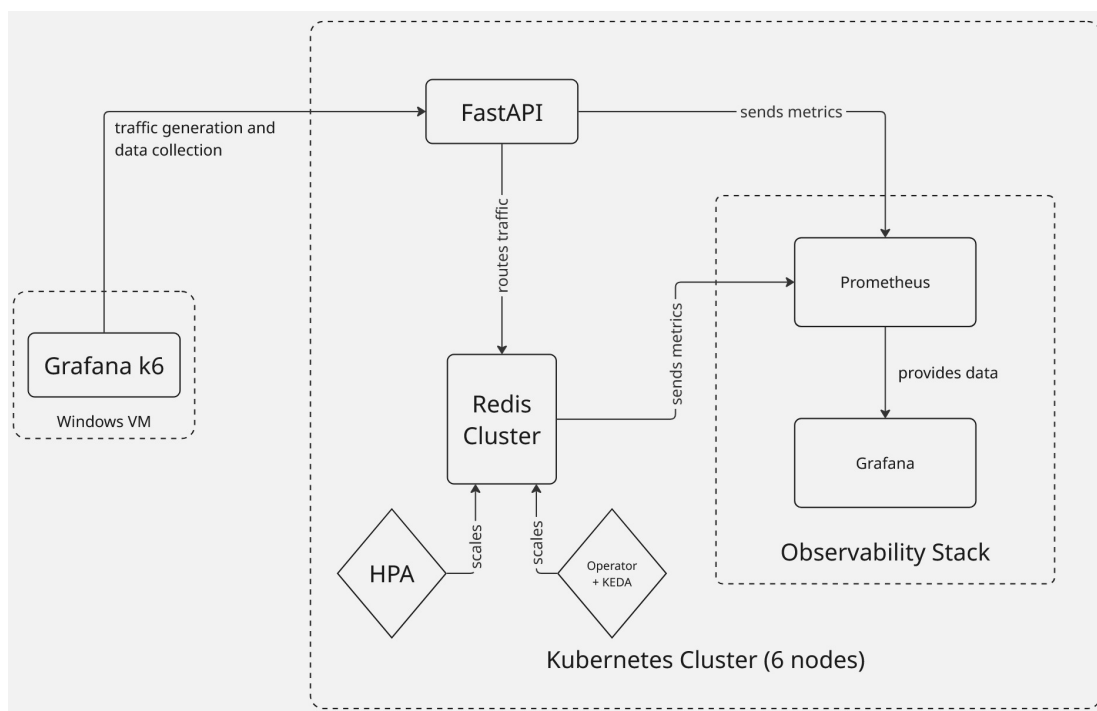


Figure 1: High-level architecture of the testbed.

5.1 Components Outside the Cluster

All synthetic load originates from a single intermediary virtual machine (192.168.1.100), which is also the host used to access the cluster remotely over the Remote Desktop

Protocol (RDP). Running the load generator off-cluster serves two purposes: it reproduces the realistic condition in which traffic arrives from outside the cluster network, and it ensures that the CPU and memory cost of generating load does not contend with the workloads being measured on the six mini-PCs.

The generator is Grafana k6, an open-source load-testing tool whose scenarios are written in JavaScript (Grafana Labs, 2024c). The test script (`load-test.js`) is parameterised so that the identical script drives both experimental runs, preserving reproducibility. Three design decisions in the script are architecturally relevant:

- **A long, steady plateau.** The script uses the `ramping-arrival-rate` executor to ramp to a constant request rate (`STEADY_RATE`, default 1000 req/s) and hold it for a configurable duration (`PLATEAU_MINUTES`, default 20). A sustained plateau is required so that the manual scale-out, settle, and scale-in steps can all be performed while traffic is live. The same rate is used in both scenarios so the workload is held constant rather than a variable.
- **Round-robin ingress across all six nodes.** Rather than targeting one address, each request is sent to one of the six node IPs (192.168.1.101–106) on the FastAPI NodePort (30080). Concentrating load on a single node saturates that node's connection-tracking table and surfaces as client-side connection timeouts; spreading requests keeps the experiment measuring Redis scaling rather than single-node ingress saturation.
- **A shardable, cache-miss-inducing workload.** The workload models a gaming leaderboard. Keys take the form `lb:<region>:<mode>:<date>` and are spread across many Redis sorted sets (no hash tags), so that adding or resharding nodes can genuinely relieve load. Traffic is split 80% writes (`POST /score/`, a sorted-set update) to 20% reads (`GET /leaderboard/`); a fixed fraction of reads deliberately query non-existent (future-dated) boards to generate key-space misses, which is the signal the KEDA scenario scales on.

Upon completion k6 emits an end-of-test summary and, throughout the run, pushes its time-series metrics directly into the Prometheus remote-write endpoint exposed by the observability stack (Section 5.2.4), so that client-side results appear on the same Grafana dashboards as the cluster-side metrics. A representative excerpt of the script is shown in Listing 1; helper functions that synthesise warm and cold board keys are omitted for brevity.

Listing 1: Excerpt of the k6 load script (`load-test.js`).

```
import http from 'k6/http';
import { check } from 'k6';
import { randomString, randomIntBetween } from
  ↪ 'https://jslib.k6.io/k6-utils/1.2.0/index.js';

// Long, steady plateau; tune via env and keep identical across
  ↪ both scenarios so that the scaling MECHANISM is the only
  ↪ variable.
const STEADY_RATE = Number(__ENV.STEADY_RATE || 1000);
const PLATEAU_MINUTES = Number(__ENV.PLATEAU_MINUTES || 20);
```

5. Experiment Architecture

```
export const options = {
  scenarios: {
    redis_scaling_trigger: {
      executor: 'ramping-arrival-rate',
      startRate: 50,
      timeUnit: '1s',
      preAllocatedVUs: 50,
      maxVUs: 300,
      stages: [
        { duration: '1m', target: STEADY_RATE },
        ↪ // ramp up
        { duration: `${PLATEAU_MINUTES}m`, target: STEADY_RATE
        ↪ }, // hold the plateau
        { duration: '30s', target: 0 },
        ↪ // ramp down
      ],
    },
  },
};

// Round-robin across ALL six NodePorts so no single node's
↪ conntrack table saturates.
const API_PORT = __ENV.API_PORT || '30080';
const DEFAULT_HOSTS = [
  '192.168.1.101', '192.168.1.102', '192.168.1.103',
  '192.168.1.104', '192.168.1.105', '192.168.1.106',
];
const BASES = DEFAULT_HOSTS.map((h) =>
  ↪ `http://${h}:${API_PORT}`);
function apiBase() { return BASES[Math.floor(Math.random() *
  ↪ BASES.length)]; }

const COLD_READ_FRACTION = 0.25; // reads that miss -> keyspace
↪ misses -> drive KEDA

export default function () {
  const base = apiBase();
  const isWrite = Math.random() < 0.8; // 80% writes, 20% reads
  if (isWrite) {
    const payload = JSON.stringify({
      board: warmBoard(), player_id: `player_${randomString(8)} `,
      score: randomIntBetween(100, 10000),
    });
    const res = http.post(`${base}/score/`, payload,
      { headers: { 'Content-Type': 'application/json' } });
    check(res, { 'score update status is 200': (r) => r.status
      ↪ === 200 });
  } else {
    const board = Math.random() < COLD_READ_FRACTION ?
      ↪ coldBoard() : warmBoard();
    const res = http.get(`${base}/leaderboard/?board=
      ${
        encodeURIComponent(board)
      }`);
    check(res, { 'leaderboard fetch status is 200': (r) =>
      ↪ r.status === 200 });
  }
}
```

The test is launched from the VM with, for example:

```
k6 run -o experimental-prometheus-rw -e
  ↪ API_URL="http://192.168.1.101:30080"
  ↪ load-generation-test.js
```

5.2 Components Inside the Cluster

The cluster is a k3s distribution running on six mini-PCs (four-core CPU, 8 GB RAM each), one acting as a combined control-plane and worker node and the remaining five as workers (K3s Project, 2024). k3s is chosen for its small footprint on resource-constrained hardware while remaining fully API-conformant. All in-cluster components are organised into four Kubernetes namespaces, which also structure the subsections below: `default` (the application entrypoint), `redis` (the stateful data tier and both scaling configurations), `ot-operators` together with `keda` (the cluster-wide scaling controllers), and `monitoring` (the observability stack). The namespace boundaries and their associated manifests follow the project’s infrastructure inventory.

5.2.1 The `default` Namespace: FastAPI Entrypoint

The `default` namespace hosts the FastAPI application that is the single ingress point for all traffic from k6 (FastAPI, 2024b). It is deployed as a Deployment of six replicas—one per physical node, so that no single pod absorbs all ingress—fronted by a NodePort Service on port 30080, which is the port the off-cluster load generator targets. The application is intentionally written to connect to *either* data-tier topology: its `REDIS_STARTUP_NODES` environment variable lists both the HPA scenario’s headless service (`redis-headless`) and the operator scenario’s leader service (`redis-cluster-leader`), and it binds to whichever currently exists. A readiness probe on `/ready` reports the pod as ready only once its Redis client is live, which doubles as a “cluster reachable” barrier for the setup scripts. The manifest is shown in Listing 2.

Listing 2: FastAPI entrypoint Deployment and NodePort Service (entrypoint-deployment.yaml).

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: fastapi-entrypoint
  labels: { app: entrypoint }
spec:
  replicas: 6                                # one per node; spreads
    ↪ ingress
  selector:
    matchLabels: { app: entrypoint }
  template:
    metadata:
      labels: { app: entrypoint }
    spec:
      containers:
        - name: fastapi
          image: dockbudy360/fastapi-entrypoint:2.5
          ports:
```

5. Experiment Architecture

```
- containerPort: 8000
env:
# Connects to whichever data-tier service currently
  ↪ exists:
#   redis-headless          -> HPA StatefulSet
#   redis-cluster-leader -> Opstree RedisCluster
- name: REDIS_STARTUP_NODES
  value:
    "redis-headless.redis.svc.cluster.local:6379,
    redis-cluster-leader.redis.svc.
    cluster.local:6379"
- name: WEB_CONCURRENCY
  value: "4"
readinessProbe:
  httpGet: { path: /ready, port: 8000 }
  initialDelaySeconds: 5
  periodSeconds: 10
livenessProbe:
  httpGet: { path: /healthy, port: 8000 }
  initialDelaySeconds: 10
  periodSeconds: 15
resources:
  requests: { cpu: "500m", memory: "256Mi" }
  limits:   { cpu: "2",      memory: "512Mi" }
---
apiVersion: v1
kind: Service
metadata:
  name: fastapi-entrypoint-svc
spec:
  type: NodePort
  selector: { app: entrypoint }
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8000
      nodePort: 30080
```

5.2.2 The redis Namespace: Stateful Data Tier and Scaling Configurations

The `redis` namespace contains the stateful data tier and the manifests for both scaling scenarios. Only one scenario is ever active at a time; the setup scripts scorch the namespace between runs. A Redis Cluster partitions its key space into 16,384 hash slots distributed across master nodes (Redis Ltd., 2024b), and the two scenarios differ precisely in who manages those slots when the node count changes.

HPA scenario (self-managed cluster). Here the cluster is assembled from native Kubernetes primitives. A `ConfigMap` supplies a minimal `redis.conf` with `cluster-enabled yes`; a headless `Service` (`clusterIP: None`) gives each pod a stable DNS identity; and a `StatefulSet` of six pods runs Redis with an `oliver006` exporter sidecar on port 9121 (Listing 3). The cluster topology—three masters, each with one replica—is not expressible in the `StatefulSet` itself and is therefore formed imperatively by the setup script with `redis-cli --cluster create --cluster-replicas 1`. A `HorizontalPodAutoscaler` (Listing 4) targets the

5. Experiment Architecture

StatefulSet on a 40% average CPU goal (The Kubernetes Authors, n.d.-a); its `minReplicas` is pinned to the six-pod baseline so the idle controller cannot remove the replicas, and a behavior block fixes the scale-up and scale-down rates so that autoscaler dynamics are held identical to the KEDA scenario. A `ServiceMonitor` (Listing 5) wires the exporter into Prometheus.

Listing 3: Redis ConfigMap, headless Service, and StatefulSet (`redis-hpa-cluster.yaml`).

```
apiVersion: v1
kind: ConfigMap
metadata: { name: redis-min-config, namespace: redis }
data:
  redis.conf: |
    cluster-enabled yes
    cluster-config-file nodes.conf
    cluster-node-timeout 5000
    appendonly no
---
apiVersion: v1
kind: Service
metadata:
  name: redis-headless
  namespace: redis
  labels: { app: redis }
spec:
  clusterIP: None                # headless: per-pod DNS for
    ↪ slot-aware routing
  ports:
    - { port: 6379, name: redis }
    - { port: 16379, name: gossip }
    - { port: 9121, name: metrics }
  selector: { app: redis }
---
apiVersion: apps/v1
kind: StatefulSet
metadata: { name: redis, namespace: redis }
spec:
  serviceName: redis-headless
  replicas: 6                    # 3 masters + 3 replicas;
    ↪ roles assigned at runtime by redis-cli
  selector:
    matchLabels: { app: redis }
  template:
    metadata:
      labels: { app: redis }
    spec:
      containers:
        - name: redis
          image: redis:7.2-alpine
          command: ["redis-server", "/etc/redis/redis.conf"]
          resources:
            requests: { cpu: "100m", memory: "128Mi" } #
              ↪ required for HPA % calc
          volumeMounts:
            - { name: conf, mountPath: /etc/redis }
        - name: redis-exporter
          image: ghcr.io/oliver006/redis_exporter:latest
          ports:
            - { containerPort: 9121, name: metrics }
          env:
```

5. Experiment Architecture

```
- { name: REDIS_ADDR, value:
  ↪ "redis://localhost:6379" }
resources:
  requests: { cpu: "50m", memory: "64Mi" }
volumes:
- name: conf
  configMap: { name: redis-min-config }
```

Listing 4: HorizontalPodAutoscaler for the HPA scenario (redis-hpa-scaling.yaml).

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: redis-hpa, namespace: redis }
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: StatefulSet
    name: redis
  minReplicas: 6 # = baseline (3 masters + 3
  ↪ replicas), so replicas are not stripped
  maxReplicas: 9 # room for the HPA to add
  ↪ ghost pods
  metrics:
  - type: ContainerResource
    containerResource:
      name: cpu
      container: redis # only the Redis container,
      ↪ not the exporter
      target:
        type: Utilization
        averageUtilization: 40
  behavior: # matched IDENTICALLY on the
  ↪ KEDA ScaledObject
  scaleDown:
    stabilizationWindowSeconds: 300
    policies: [{ type: Pods, value: 1, periodSeconds: 60 }]
  scaleUp:
    stabilizationWindowSeconds: 0
    policies: [{ type: Pods, value: 1, periodSeconds: 180 }]
```

Listing 5: ServiceMonitor for the HPA scenario exporter (redis-servicemonitor.yaml).

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: redis-monitor
  namespace: redis
  labels: { release: prometheus } # adopted by
  ↪ kube-prometheus-stack
spec:
  selector:
    matchLabels: { app: redis }
  endpoints:
  - { port: metrics, interval: 15s }
```

Operator and KEDA scenario (operator-managed cluster). Here the data tier is declared rather than assembled. The Opstree Redis Operator is given a desired

5. Experiment Architecture

clusterSize through its RedisCluster custom resource, and the operator creates the leader and follower StatefulSets, forms the cluster, and—crucially—reshards slots when the size changes (The Kubernetes Authors, n.d.-b; OpsTree Solutions, n.d.). The resource is deployed via the operator’s Helm chart, configured by the values file in Listing 6: clusterSize is 3, the leader and follower replica counts are left unset so that clusterSize alone drives the master count, leader anti-affinity spreads masters across nodes, and the same oliver006 exporter is enabled for metric parity with the HPA scenario. Scaling is actuated by a KEDA ScaledObject (Listing 7) that reads a Prometheus cache-miss-ratio query and changes clusterSize through a managed HPA bound to the resource’s /scale subresource (KEDA Project, 2024a); its behavior block mirrors the HPA scenario exactly. As in the HPA scenario, a standalone ServiceMonitor (Listing 8) is required, because neither the operator nor the chart flag creates one.

Listing 6: Opstree Helm values producing the RedisCluster (redis-operator-cluster-values.yaml).

```
redisCluster:
  clusterSize: 3
  persistenceEnabled: false          # ephemeral, matching the HPA
  ↪ scenario
  resources:
    requests: { cpu: "100m", memory: "128Mi" }
    limits:   { cpu: "250m", memory: "256Mi" }
  leader:
    replicas: null                    # null => clusterSize drives
    ↪ the master count
    affinity:
      podAntiAffinity:                # prefer one master per node
        preferredDuringSchedulingIgnoredDuringExecution:
          - weight: 100
      podAffinityTerm:
        labelSelector:
          matchLabels: { app: redis-cluster-leader }
        topologyKey: kubernetes.io/hostname
  follower:
    replicas: null
redisExporter:
  enabled: true
  image: ghcr.io/oliver006/redis_exporter
  tag: latest
  resources:
    requests: { cpu: "50m", memory: "64Mi" }
serviceMonitor:                       # chart flag is a no-op here;
  ↪ see standalone SM below
  enabled: true
  interval: 15s
  extraLabels: { release: prometheus }
```

Listing 7: KEDA ScaledObject with the cache-miss-ratio trigger (redis-keda-scaling.yaml).

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata: { name: redis-keda-scaler, namespace: redis }
spec:
  scaleTargetRef:                      # the operator’s CR, NOT a
  ↪ StatefulSet
```

5. Experiment Architecture

```
    apiVersion: redis.redis.opstreelabs.in/v1beta2
    kind: RedisCluster
    name: redis-cluster
    minReplicaCount: 3
    maxReplicaCount: 6
    advanced:
      restoreToOriginalReplicaCount: true
      horizontalPodAutoscalerConfig:
        behavior: # IDENTICAL to the HPA's
          ↪ behavior block
        scaleDown:
          stabilizationWindowSeconds: 300
          policies: [{ type: Pods, value: 1, periodSeconds: 60 }]
        scaleUp:
          stabilizationWindowSeconds: 0
          policies: [{ type: Pods, value: 1, periodSeconds: 180
            ↪ }]
    triggers:
      - type: prometheus
        metricType: Value
        metadata:
          serverAddress:
            ↪ http://prometheus-kube-prometheus-prometheus.
            ↪ monitoring.svc.cluster.local:9090
        query: |
          sum(rate(redis_keyspace_misses_total
            ↪ {namespace="redis"}[1m]))
          /
          (sum(rate( {namespace="redis"}[1m])) +
            ↪ sum(rate(redis_keyspace_misses_total
            ↪ {namespace="redis"}[1m])))
        threshold: "0.05"
```

Listing 8: Standalone ServiceMonitor for the operator's exporters (redis-servicemonitor.yaml).

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: redis-operator-monitor
  namespace: redis
  labels: { release: prometheus }
spec:
  selector: # selects both leader-metrics
    ↪ and follower-metrics services
  matchLabels:
    app.kubernetes.io/component: metrics
    redis_setup_type: cluster
  endpoints:
    - { port: redis-exporter, interval: 15s }
```

5.2.3 The ot-operators and keda Namespaces: Scaling Controllers

The operator scenario depends on two cluster-wide controllers that are installed once, via Helm, and shared across runs; they are not redeployed by the per-experiment scripts. The ot-operators namespace runs the Opstree Redis Operator, a custom controller that watches RedisCluster resources and reconciles them into running

clusters, encoding the Redis-specific domain knowledge—cluster formation, slot assignment, and topology-aware resharding—that Kubernetes’ generic primitives lack (The Kubernetes Authors, n.d.-b). The `keda` namespace runs the KEDA controller, which materialises a managed `HorizontalPodAutoscaler` from each `ScaledObject` and exposes the external Prometheus metric to it (KEDA Project, 2024a). In this design the `ScaledObject` itself (Listing 7) lives in the `redis` namespace alongside its target, while the controller that acts on it is cluster-wide. The HPA scenario uses neither controller; its autoscaler is the built-in Kubernetes HPA controller, requiring no additional namespace.

5.2.4 The monitoring Namespace: Observability Stack

The `monitoring` namespace hosts the observability stack, deployed once from the `kube-prometheus-stack` Helm chart, which bundles Prometheus and Grafana (Prometheus Community, 2024a). It is identical across both experimental conditions. The values overrides are shown in Listing 9: control-plane scrape jobs that do not apply to a `k3s` distribution are disabled; both Prometheus and Grafana are exposed via `NodePort`; and, critically, the Prometheus `enableRemoteWriteReceiver` flag is set so that the off-cluster `k6` generator can push its client-side metrics into the same Prometheus instance.

Prometheus discovers its in-cluster scrape targets through the `ServiceMonitor` resources defined in the `redis` namespace (Listings 5 and 8), which point it at the Redis exporters, and through the FastAPI service’s instrumentation endpoint (Prometheus Authors, 2024). The same Prometheus instance serves a dual role: it is both the visualisation backend for Grafana (Grafana Labs, 2024b) and the external metric source that the KEDA cache-miss-ratio trigger queries, creating a single unified data path from application telemetry to autoscaling decisions.

Listing 9: `kube-prometheus-stack` Helm values (`my-prometheus-values.yaml`).

```
kubeControllerManager: { enabled: false }    # not exposed by k3s
kubeEtcd:                { enabled: false }
kubeProxy:               { enabled: false }
kubeScheduler:          { enabled: false }
grafana:
  service: { type: NodePort }
prometheus:
  service: { type: NodePort }
  prometheusSpec:
    enableRemoteWriteReceiver: true          # accept k6
    ↪ remote-write
```

5.3 End-to-End Data Flow

Bringing the layers together, a single iteration during a test run traverses the path illustrated in Figure 1:

1. `k6` on the intermediary VM issues an HTTP request to one of the six node IPs on `NodePort` 30080, which routes to a FastAPI endpoint pod.

5. Experiment Architecture

2. The FastAPI handler hashes the leaderboard key and issues the corresponding sorted-set command to the responsible Redis master, resolved through the data-tier service; it returns the result to k6.
3. The Redis exporter sidecars and the FastAPI instrumentation expose metrics, which Prometheus scrapes on its interval; k6 concurrently remote-writes its own client-side metrics to Prometheus.
4. The active scaling module consumes those metrics: the HPA reads CPU utilisation from the metrics API, whereas KEDA reads the Prometheus cache-miss-ratio. Each computes a desired size and acts—scaling a role-blind `StatefulSet` in the HPA case, or changing `clusterSize` (and thereby triggering an operator reshard) in the KEDA case.
5. Grafana queries Prometheus and renders both cluster-side and client-side series on shared dashboards, providing the observations recorded in Section 7.

Every component on this path other than the scaling module in step 4 is held constant across the two experimental conditions, so the comparison in Section 7 isolates the scaling mechanism.

6 Experimental Test Flow

The evaluation contrasts two mechanisms for autoscaling a sharded, stateful Redis Cluster on the same six-node k3s cluster: (i) the native Kubernetes *Horizontal Pod Autoscaler* (HPA) acting on a self-managed cluster, and (ii) a domain-specific *operator* driven by *Kubernetes Event-driven Autoscaling* (KEDA). Both scenarios are exercised with an identical, deterministic six-phase procedure so that the scaling mechanism is the sole independent variable: (1) scorch all prior state and scale the FastAPI endpoint to zero; (2) provision the namespace, the Redis Cluster, and the endpoint; (3) deploy the autoscaler; (4) drive a controlled k6 workload and observe the scale-out; (5) quiesce the workload and perform a controlled scale-in; and (6) capture the resulting topology and key-space metrics in Grafana. The workload, resource requests, monitoring stack, and autoscaler behavior parameters are held constant; only the resharding intelligence differs. Bash scripts help to automate most of the procedure. For more details on the Bash scripts, please see the accompanying repository.

A Redis Cluster partitions its key space into 16,384 hash slots distributed across master nodes, so adding or removing a master requires *resharding*—the live migration of slots between nodes—rather than the mere addition of a pod (Redis, n.d.; Redis Ltd., 2024b). The central question of the test flow is whether each mechanism can perform this slot-level work, because Kubernetes’ native workload primitives manage *pod count* but possess no knowledge of cluster *topology* (The Kubernetes Authors, n.d.-b).

6.1 Scenario A: Native HPA on a Self-Managed Redis Cluster

1. **Provisioning.** A StatefulSet of six Redis pods is applied and the cluster is formed imperatively with `redis-cli --cluster create --cluster-replicas 1`, yielding three masters (`redis-0..2`) each with one replica (`redis-3..5`). Because a StatefulSet manifest cannot express the master-replica relationship, this role assignment is necessarily a runtime operation external to the declarative configuration (Redis Ltd., 2024b; The Kubernetes Authors, n.d.-c).
2. **Autoscaler.** An HPA targets the StatefulSet with a 40% average CPU-utilisation goal; per-pod CPU requests are declared so the controller can compute utilisation (The Kubernetes Authors, n.d.-a). `minReplicas` is pinned to the six-pod baseline so the idle controller cannot scale down and strip the replicas before the run.
3. **Scale-out.** The workload drives master CPU past the target. The HPA raises the desired replica count according to $\text{desired} = \lceil \text{current} \times (\text{currentMetric} / \text{targetMetric}) \rceil$ (The Kubernetes Authors, n.d.-a) and appends pods beginning at `redis-6`. These pods join the StatefulSet but are never issued a `CLUSTER MEET` nor assigned slots, so each remains an isolated single-node island—a *ghost pod* owning zero hash slots and serving no traffic.
4. **Scale-in.** The HPA is removed and the StatefulSet is scaled down one pod at a time. StatefulSets terminate pods in reverse-ordinal order, from $\{N-1 \dots 0\}$ (The Ku-

bernetes Authors, n.d.-c), so the three replicas (`redis-5,4,3`) are deleted first; native cluster failover preserves every slot and no data is lost. The subsequent deletion removes a master (`redis-2`) whose replica is already gone. With no drain, no reshard, and no surviving replica to promote, that master's slots are orphaned and cluster-wide coverage falls below 16,384.

Hypothesis. The HPA will provision capacity it structurally cannot integrate on scale-out: ghost pods accumulate while throughput and latency fail to recover, because the three original masters stay saturated. On scale-in it will inflict irreversible data loss once redundancy is exhausted—the reverse-ordinal *data cliff*, observable as `cluster_state:fail` and a step decrease in total keys. Crucially, the replicas will *delay* but not *prevent* the cliff, demonstrating that failover (protection against unplanned node death) is no substitute for the slot migration a planned scale-in requires.

6.2 Scenario B: Operator and KEDA on an Operator-Managed Redis Cluster

1. **Provisioning.** The Opstree Redis Operator—a custom controller and `RedisCluster` custom resource that encode Redis-specific domain knowledge into a Kubernetes reconciliation loop (OpsTree Solutions, n.d.; The Kubernetes Authors, n.d.-b)—is given a desired `clusterSize`. It declaratively creates the leader and follower `StatefulSets` and forms the cluster itself, issuing `CLUSTER MEET` and assigning slots, with no manual `redis-cli` step.
2. **Autoscaler.** A KEDA `ScaledObject` watches a Prometheus cache-miss-ratio metric and actuates the operator's `clusterSize` through a managed HPA bound to the custom resource's `/scale` subresource (KEDA Project, 2024a).
3. **Scale-out.** When the metric crosses its threshold, KEDA raises `clusterSize`; the operator adds a master *and* performs a topology-aware reshard, migrating a balanced slot range onto the new node so that the added capacity actually relieves load (Redis, n.d.).
4. **Scale-in.** For reproducibility the cluster is first reset to a settled three-master baseline, then transitioned a single master at a time ($3 \rightarrow 4 \rightarrow 3$) via KEDA's `paused-replicas` annotation. On removal the operator drains the departing master—migrating its slots to the survivors—before deleting the pod. A documented limitation is captured: the operator has no self-healing for an interrupted reshard, which leaves a slot in the `importing/migrating` (“open”) state (Redis Ltd., 2024b) that must be repaired manually with `redis-cli --cluster fix`.

Hypothesis. The operator will integrate each new master on scale-out (full slot ownership and recovering throughput) and will drain gracefully on scale-in, holding continuous 16,384-slot coverage with zero data loss. This safety is expected to come at the cost of substantially more provisioning machinery than the HPA path and a residual need

for manual intervention when a reshard is interrupted—supporting the thesis that the operator is *necessary but not, on its own, sufficient* for fully hands-off elastic operation.

6.3 The Necessity of Quiescing the Load Generator Before Scale-In

Resharding is performed live and depends on clients following ASK and MOVED redirections while a slot is held in the `importing/migrating` state (Redis, n.d.; Redis Ltd., 2024b). Under the write-heavy workload, a continuous stream of writes addressed to a slot that is mid-migration is the dominant cause of an interrupted migration, leaving an “open” slot that the operator cannot repair on its own. Performing the controlled scale-in while the generator is still firing therefore introduces a confound: an interruption attributable to write pressure would be indistinguishable from a deficiency of the scaling mechanism itself.

Because this study is scoped to the ghost-pod phenomenon and the operator’s resharding *capability*—not its robustness under adversarial write load—the controlled scale-in is conducted under quiescent conditions. The `k6` generator is terminated and the procedure continues only once the request rate, and the corresponding Redis command rate, have fallen to zero. This is a deliberate experimental control rather than a concession for three reasons. First, it isolates resharding correctness from a known confounding stressor. Second, it remains fair to both scenarios, as the HPA data cliff is structural and manifests even at zero load, so quiescing cannot conceal it. Third, it preserves measurability: the workload configures no key expiration or eviction, so populated keys persist for post-hoc accounting of any loss. The load-induced interruption is not discarded but reported separately as a finding of the scale-out experiments.

Hypothesis. Quiescing the generator before the scale-in will sharply reduce the probability of an interrupted reshard, yielding a reproducible graceful drain for the operator while leaving the HPA cliff unchanged. This isolates write load—rather than the scaling mechanism alone—as the governing factor in reshard interruption, and so permits a balanced comparison of operational complexity between the two approaches.

7 Results and Analysis

8 Conclusion

Anhang

Hier steht der Anhang.

Glossar

Hier steht das Glossar.

Literaturverzeichnis

- Amazon Web Services. (2022). *Container build lens — AWS well-architected framework: Design principles* [Accessed: 2026-01-24]. <https://docs.aws.amazon.com/pdfs/wellarchitected/latest/container-build-lens/container-build-lens.pdf#design-principles>
- Cloud Native Computing Foundation. (2023). *CNCF announces graduation of KEDA* [Accessed: 2026-01-24]. <https://www.cncf.io/announcements/2023/08/22/cloud-native-computing-foundation-announces-graduation-of-kubernetes-autoscaler-keda/>
- Cloud Native Computing Foundation. (2024). *Managing large-scale Redis clusters on Kubernetes with an operator* [Accessed: 2026-01-24]. <https://www.cncf.io/blog/2024/12/17/managing-large-scale-redis-clusters-on-kubernetes-with-an-operator-kuaishous-approach/>
- CNCF TAG App Delivery. (2023). *CNCF operator white paper* [Accessed: 2026-01-24]. <https://tag-app-delivery.cncf.io/whitepapers/operator/>
- Docker Inc. (2016). *Containers are not VMs* [Accessed: 2026-01-24]. <https://www.docker.com/blog/containers-are-not-vm/>
- FastAPI. (2024a). *Concurrency and async / await — FastAPI* [Accessed: 2026-01-24]. <https://fastapi.tiangolo.com/async/>
- FastAPI. (2024b). *FastAPI — modern, fast, web framework for building APIs with python* [Accessed: 2026-01-24]. <https://fastapi.tiangolo.com/>
- Google Cloud. (2023). *Zero-downtime scaling in Memorystore for Redis cluster* [Accessed: 2026-01-24]. <https://cloud.google.com/blog/products/databases/zero-downtime-scaling-in-memorystore-for-redis-cluster>
- Grafana Labs. (2024a). *API load testing — Grafana k6 documentation* [Accessed: 2026-01-24]. <https://grafana.com/docs/k6/latest/testing-guides/api-load-testing/>
- Grafana Labs. (2024b). *Grafana — the open and composable observability platform* [Accessed: 2026-01-24]. <https://github.com/grafana/grafana>
- Grafana Labs. (2024c). *Grafana k6 documentation* [Accessed: 2026-01-24]. <https://grafana.com/docs/k6/latest/>
- Grafana Labs. (2024d). *K6 — a modern load testing tool* [Accessed: 2026-01-24]. <https://github.com/grafana/k6>
- Helm Project. (2024). *Using Helm* [Accessed: 2026-01-24]. https://helm.sh/docs/intro/using_helm/
- IBM. (2024). *The history of kubernetes* [Accessed: 2026-01-24]. <https://www.ibm.com/think/topics/kubernetes-history>
- K3s Project. (2024). *K3s — lightweight kubernetes* [Accessed: 2026-01-24]. <https://docs.k3s.io/>
- KEDA Authors. (2024a). *Keda – kubernetes event-driven autoscaling* [Accessed: 2026-01-24]. <https://keda.sh/>
- KEDA Authors. (2024b). *Keda concepts* [Accessed: 2026-01-24]. <https://keda.sh/docs/2.19/concepts/>
- KEDA Project. (2024a). *KEDA — kubernetes event-driven autoscaling* [Accessed: 2026-01-24]. <https://keda.sh/>

- KEDA Project. (2024b). *KEDA — prometheus scaler* [Accessed: 2026-01-24]. <https://keda.sh/docs/2.19/scalers/prometheus/>
- KEDA Project. (2024c). *KEDA — ScaledObject specification* [Accessed: 2026-01-24]. <https://keda.sh/docs/2.19/reference/scaledobject-spec/>
- KEDA Project. (2024d). *KEDA — scaling deployments, StatefulSets & custom resources* [Accessed: 2026-01-24]. <https://keda.sh/docs/2.19/concepts/scaling-deployments/>
- Kubernetes Documentation. (2024a). *Autoscaling workloads* [Accessed: 2026-01-24]. <https://kubernetes.io/docs/concepts/workloads/autoscaling/>
- Kubernetes Documentation. (2024b). *Disruptions — Pod disruption budgets* [Accessed: 2026-01-24]. <https://kubernetes.io/docs/concepts/workloads/pods/disruptions/>
- Kubernetes Documentation. (2024c). *Horizontal pod autoscaling* [Accessed: 2026-01-24]. <https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/>
- Kubernetes Documentation. (2024d). *Kubernetes overview* [Accessed: 2026-01-24]. <https://kubernetes.io/docs/concepts/overview/>
- Kubernetes Documentation. (2024e). *Operator pattern* [Accessed: 2026-01-24]. <https://kubernetes.io/docs/concepts/extend-kubernetes/operator/>
- Kubernetes Documentation. (2024f). *Statefulsets* [Accessed: 2026-01-24]. <https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/>
- Kubernetes Documentation. (2024g). *Statefulsets* [Accessed: 2026-01-24]. <https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/>
- Medium / emreblblvv. (2024). *Auto-scaling with KEDA using custom RED metrics from Prometheus* [Accessed: 2026-01-24]. <https://medium.com/@emreblblvv/auto-scaling-with-keda-using-custom-red-metrics-from-prometheus-76d50785e442>
- Mell, P., & Grance, T. (2011). *The NIST definition of cloud computing* (tech. rep. No. Special Publication 800-145) (Accessed: 2026-01-24). National Institute of Standards and Technology. <https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-145.pdf>
- OpsTree Solutions. (n.d.). *Redis-operator: A Golang based Redis operator for Kubernetes* [Accessed: 2026-06-06].
- Portworx. (2024). *Understanding StatefulSets in Kubernetes* [Accessed: 2026-01-24]. <https://portworx.com/knowledge-hub/understanding-statefulsets-in-kubernetes/>
- Prometheus Authors. (2024). *Prometheus overview* [Accessed: 2026-01-24]. <https://prometheus.io/docs/introduction/overview/>
- Prometheus Community. (2024a). *Kube-prometheus-stack Helm chart* [Accessed: 2026-01-24]. <https://artifacthub.io/packages/helm/prometheus-community/kube-prometheus-stack>
- Prometheus Community. (2024b). *Kube-prometheus-stack helm chart* [Accessed: 2026-01-24]. <https://github.com/prometheus-community/helm-charts>
- Red Hat. (2025). *Stateful vs. stateless applications* [Accessed: 2026-01-24]. <https://www.redhat.com/en/topics/cloud-native-apps/stateful-vs-stateless>

- Redis. (n.d.). *Scale with redis cluster* [Accessed: 2026-06-06]. https://redis.io/docs/latest/operate/oss_and_stack/management/scaling/
- Redis Ltd. (2024a). *Redis — real-time data for agents & apps* [Accessed: 2026-01-24]. <https://redis.io/>
- Redis Ltd. (2024b). *Redis cluster specification* [Accessed: 2026-01-24]. https://redis.io/docs/latest/operate/oss_and_stack/reference/cluster-spec/
- The Kubernetes Authors. (n.d.-a). *Horizontal pod autoscaling* [Accessed: 2026-06-06]. <https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/>
- The Kubernetes Authors. (n.d.-b). *Operator pattern* [Accessed: 2026-06-06]. <https://kubernetes.io/docs/concepts/extend-kubernetes/operator/>
- The Kubernetes Authors. (n.d.-c). *Statefulsets* [Accessed: 2026-06-06]. <https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/>
- Thoras. (2024). *Predictive horizontal pod autoscaling* [Accessed: 2026-01-24]. <https://docs.thoras.ai/guides/hpa>
- Vazquez, A. (2026). *Kubernetes HPA best practices: When CPU works, why memory doesn't* [Accessed: 2026-01-24]. <https://alexandre-vazquez.com/kubernetes-hpa-best-practices/>
- Wanigasooriya, C., & Ekanayake, I. (2026). *NimbusGuard: A novel framework for proactive Kubernetes autoscaling using deep Q-networks* [Accessed: 2026-01-24]. *arXiv preprint arXiv:2604.11017*. <https://arxiv.org/html/2604.11017v1>

Stichwortverzeichnis

Hier steht das Stichwortverzeichnis.

Anlagen

Hier sind die Anlagen.